OPERATING SYSTEMS & PARALLEL COMPUTING

Dual Degree in Business Administration & Data and Business Analytics
BBADBA SEP-2026 OSPC-NBDA.3.M.B

Area Computer Science
Number of sessions: 30
Academic year: 26-27
Degree course: THIRD
Number of credits: 6.0
Semester: 1º
Category: COMPULSORY
Language: English

Professor: **MATTEO TURILLI**

E-mail: mturilli@faculty.ie.edu

Dr. Matteo Turilli is an Associate Professor in the bachelor's program in Computer Science and Artificial Intelligence at IE University and serves as Chief Laboratories Office at the IE School of Science and Technology. Before joining IE, he was an Associate Research Professor at Rutgers University, NJ, USA, and an Advanced Application Engineer at the National Brookhaven Laboratory, Department of Energy, NY, USA. Before moving to Rutgers, Dr. Turilli was the Chair of the EGI Federated Cloud and a Senior Research Associate at the Oxford e-Research Centre (OeRC), University of Oxford, UK. Dr. Turilli holds a PhD in Computer Science from the University of Oxford, UK, and has published more than a hundred papers in peer-reviewed conference proceedings and journals. His research interests include the intersection of machine learning and high-performance computing (HPC), novel design abstractions for HPC and cloud computing platforms, performance characterization of scientific workflows at scale, software sustainability, and Computer Ethics. Dr. Turilli has thirty years of experience in the private sector, including working for start-ups, funding and leading companies, and consulting for several Fortune 500 companies.

Office Hours

Office hours will be on request. Please contact at:

matteo.turilli@faculty.ie.edu

SUBJECT DESCRIPTION

Every data analysis a business analyst performs, from a pandas transformation or a scikit-learn model to a Monte Carlo simulation, runs on an operating system and, increasingly, on parallel hardware. This hardware may include multicore processors, multiprocessor servers, computing clusters, and GPU accelerators. Understanding these systems helps analysts explain why one workload runs in seconds while a slightly larger one takes hours, why adding processors can slow a program, and when moving a workload to a cluster or GPU is worth the additional complexity.

This course builds that understanding through theory and Python examples in data management and analysis. The first half focuses on operating systems. Students study processes, threads, memory, storage, and input/output, and how these resources affect analytical workloads. They also learn to work in a Unix/Linux environment using the terminal, the standard environment for servers, clusters, and cloud computing. Using standard tools, students observe running programs and diagnose common performance problems, such as insufficient memory, slow input/output, and competition for processor resources.

The second half focuses on parallel computing. Students study the architecture of modern parallel systems, including multicore processors, shared-memory and distributed-memory systems, computing clusters, and GPU accelerators. They then use Python tools from the data analytics ecosystem to implement and evaluate parallel workloads. These tools include threading and multiprocessing, `concurrent.futures`, `joblib`, NumPy vectorization, Dask for partitioned and distributed data processing, and CuPy for GPU computing.

Students also develop a practical framework for evaluating performance. They study execution time, speedup, efficiency, overhead, Amdahl's Law, and strong and weak scaling. Rather than treating these concepts as theory only, students test them by measuring their own programs, validating their results, and explaining why performance does or does not improve.

Practical work is conducted on students' laptops and on the University's high-performance computing cluster. The course culminates in a group project in which students select a realistic analytics workload, establish a correct sequential baseline, develop a parallel implementation, run it on the cluster, measure its scaling behavior, and explain the technical and practical significance of the results.

LEARNING OBJECTIVES

The objective of this course is to provide students with a practical understanding of operating systems and parallel computing for data analysis and management. The course covers operating-system roles and structure, processes and threads, CPU scheduling, memory, storage, input/output, Linux command-line tools,

concurrency, parallel architectures, parallel programming in Python, performance and scalability, distributed data processing, and GPU computing.

At the end of the course, students should be able to:

- Explain the role and principal components of a modern operating system and how they affect the execution and performance of data-analysis workloads.
- Work in a Linux environment by navigating the file system, managing files and permissions, combining commands, automating simple tasks, and inspecting running processes and resource usage.
- Use standard operating-system tools to identify and diagnose common performance problems, including insufficient memory, input/output bottlenecks, and competition for processor resources.
- Differentiate between concurrency and parallelism, determine whether a workload is primarily CPU- or input/output-bound, and select an appropriate execution strategy.
- Describe the architecture of modern parallel systems, including multicore processors, shared-memory and distributed-memory systems, computing clusters, and GPU accelerators, and explain how these architectures affect program performance.
- Transform a correct sequential data-analysis program into a parallel implementation using appropriate Python tools, including multiprocessing, concurrent.futures, joblib, NumPy, Dask, and CuPy.
- Validate that sequential and parallel implementations produce equivalent results and identify errors caused by decomposition, shared state, task ordering, or nondeterministic execution.
- Measure and analyze the performance of parallel programs by computing speedup, efficiency, and overhead, applying Amdahl's Law, and distinguishing between strong and weak scaling.
- Process datasets that do not fit in the memory of a single process and distribute computation across the cores and nodes of a computing cluster.
- Explain when GPU acceleration is appropriate for an analytical workload, identify the costs of data transfer and initialization, and use a high-level Python interface to execute suitable operations on a GPU.
- Design, implement, deploy, and evaluate a parallel solution to a realistic analytics problem, starting with a correct sequential baseline and ending with a reproducible performance and scalability analysis on a high-performance computing cluster.

- Communicate the design decisions, experimental evidence, limitations, and practical value of a parallel computing solution to technical and business audiences.

TEACHING METHODOLOGY

IE University teaching method is defined by its collaborative, active, and applied nature. Students actively participate in the whole process to build their knowledge and sharpen their skills. Professor's main role is to lead and guide students to achieve the learning objectives of the course. This is done by engaging in a diverse range of teaching techniques and different types of learning activities.

Learning Activity	Weighting	Estimated time a student should dedicate to prepare for and participate in
Lectures	28.0 %	42.0 hours
Discussions	9.3 %	14.0 hours
Exercises in class, Asynchronous sessions, Field Work	9.3 %	14.0 hours
Group work	26.7 %	40.0 hours
Individual studying	26.7 %	40.0 hours
TOTAL	100.0 %	150.0 hours

AI POLICY

The use of generative artificial intelligence (GenAI) is allowed in this course, with the goal of developing an informed, critical perspective on its potential uses and outputs. GenAI use must be acknowledged for both text and code. Failure to acknowledge GenAI use will be treated as plagiarism. Use of GenAI will not influence the given grade.

Be aware of the limits of GenAI in its current state of development:

- If you provide minimum effort prompts, you will get low quality results. You will need to refine your prompts to get good outcomes. This will take work.
- Don't take ChatGPT's or any GenAI's output at face value. Assume it is wrong unless you either know the answer or can cross-check it with another source. You are responsible for any errors or omissions. You will be able to validate the outputs of GenAI for topics you understand.
- AI is a tool, but one that you need to acknowledge using. Failure to do so is in violation of academic honesty policies. Acknowledging the use of AI will not impact your grade.

Mandatory format to acknowledge the use of generative AI tools:

- Text. Quote the generated output and add the following note: This text was generated by [AI system(s) name/link]. The prompts used include [list of prompts].
- Code block. Add a comment before the block that includes the following note: This following code block was generated by [AI system(s) name/link]. The prompts used include [list of prompts].

If you have chosen not to include any AI generated content in your assignment, the following disclosure is recommended:

- No content generated by AI technologies has been used in this assignment.

PROGRAM

SESSION 1 (LIVE IN-PERSON)

Topics:

- How this course works: sessions, labs, assessments, and an overview of the group project
- Why the machine matters: performance differences you have already experienced, explained.
- What sits underneath a Jupyter notebook: processor, memory, storage, and the operating system.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapters 1–2 (See Bibliography)*
- *High Performance Python: Chapter 1 (See Bibliography)*

SESSION 2 (LIVE IN-PERSON)

Topics:

- The Linux operating system and the terminal.
- The Linux directory structure and navigating the file system.
- Essential commands for working with files and inspecting data from the command line.

- Exercises.

Required Reading:

- *The Linux Command Line: Part I, "Learning the Shell" (See Bibliography)*

SESSION 3 (LIVE IN-PERSON)

Topics:

- Combining commands: pipes and redirection as a data pipeline.
- Automating work: shell scripts, scheduled and background jobs, services and daemons.
- Connecting the shell and Python.
- Exercises.

Required Reading:

- *The Linux Command Line: Part II, "Configuration and the Environment" (See Bibliography)*

SESSION 4 (LIVE IN-PERSON)

Topics:

- Processes: what a program becomes when it runs.
- Observing live processes with tools such as `ps`, `top` and `htop`.
- Creating and inspecting processes from Python.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapters 4–6 (See Bibliography)*

SESSION 5 (LIVE IN-PERSON)

Topics:

- The operating system kernel and system calls.
- User mode versus kernel mode: why the boundary exists.

- Watching a program talk to the operating system: what really happens when Python reads a file.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapter 6 (See Bibliography)*

SESSION 6 (LIVE IN-PERSON)

Topics:

- Memory management: address spaces and virtual memory.
- Paging: how the operating system creates the illusion of abundant memory.
- Reading memory usage correctly, and measuring what your data really costs.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapter 13-15 (See Bibliography)*

SESSION 7 (LIVE IN-PERSON)

Topics:

- When your dataset does not fit: swapping, thrashing, and the out-of-memory killer.
- Diagnosing memory problems in practice.
- Working with limited memory in Python: efficient types, chunked processing, and related techniques.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapter 21-22 (See Bibliography)*

SESSION 8 (LIVE IN-PERSON)

Topics:

- Storage and files: attributes, operations, file types and structure.
- File permissions.

- The storage hierarchy and its consequences for loading data.
- Choosing file formats for analytics: text versus binary columnar formats.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapter 39-40 (See Bibliography)*

SESSION 9 (LIVE IN-PERSON)

OS Lab I:

- Hands-on lab session consolidating the material so far: observing processes, diagnosing memory pressure, building command-line data pipelines, and working with permissions.
- Detailed lab notebook provided; practice on your own PC.
- Lab Report 1 (individual) is assigned in this session; submission details will be announced in class.

SESSION 10 (LIVE IN-PERSON)

Topics:

- CPU scheduling: how the operating system shares the processor.
- Scheduling policies and priorities in practice.
- A first look at concurrency: what can go wrong when programs share data.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapter 7 (See Bibliography)*

SESSION 11 (LIVE IN-PERSON)

Topics:

- Security: users, groups, and permissions in multi-user systems.
- Remote computing: SSH and key-based authentication.
- Getting started on the University's high-performance computing cluster.

Required Reading:

- *Operating Systems: Three Easy Pieces: security chapters, selected sections (See Bibliography)*

SESSION 12 (LIVE IN-PERSON)

OS Lab II:

- Hands-on lab session on the University's HPC cluster: building a small automated data pipeline combining the shell and Python, and monitoring its resource usage while it runs.
- Detailed lab notebook provided.

SESSION 13 (LIVE IN-PERSON)

Topics:

- Multiple processor systems: multicore processors, caches, and large shared-memory machines.
- Why modern hardware requires parallel software.
- Synthesis: how the operating system concepts from this part of the course will reappear in parallel computing.

Required Reading:

- *Operating Systems: Three Easy Pieces: Chapter 10 (See Bibliography)*

SESSION 14 (LIVE IN-PERSON)

Review Session

- Kahoot in class simulating exam questions.
- Doubts and discussions for the exam.

SESSION 15 (LIVE IN-PERSON)

Midterm exam

SESSION 16 (LIVE IN-PERSON)

Topics:

- Measure before you optimise: profiling Python code for time and memory.
- Finding the real bottleneck, and why the answer is often surprising.

Group project launch:

- Teams, topics, and expectations.
- Each team to adopt one technique related to the course but not covered in it, and teach it to the class in the final presentation.
- Team proposal due before S18: one page, feedback only

Required Reading:

- *High Performance Python: profiling chapters (See Bibliography)*

SESSION 17 (LIVE IN-PERSON)

Topics:

- Parallel computer architecture: shared-memory and distributed-memory systems, clusters, and accelerators.
- Case study: the architecture of the University's HPC cluster.
- The fundamental cost model: computing is cheap, moving data is expensive.

Required Reading:

- *Parallel Programming for Multicore and Cluster Systems: Chapter 2, selected sections (See Bibliography)*

SESSION 18 (LIVE IN-PERSON)

Topics:

- Threads in Python and the Global Interpreter Lock.
- Race conditions, locks, and deadlocks.
- When threads help: I/O-bound workloads in data analytics.

Required Reading:

- *Parallel Programming for Multicore and Cluster Systems: Chapters 26–28 and 32 (See Bibliography)*

SESSION 19 (LIVE IN-PERSON)

Topics:

- Multiprocessing: true parallelism for CPU-bound work in Python.
- Process pools with the standard library.
- The costs of parallelism: startup, data transfer, and memory.

Required Reading:

- *High Performance Python: multiprocessing chapter (See Bibliography)*

SESSION 20 (LIVE IN-PERSON)

Topics:

- The parallelism already inside your tools: parallel scikit-learn and what `n_jobs` really does.
- NumPy vectorisation and multi-threaded numerical libraries.
- Choosing the right layer, and avoiding common pitfalls when layers combine.

Required Reading:

- *High Performance Python: matrix and vector computation chapter (See Bibliography)*

SESSION 21 (LIVE IN-PERSON)

Parallel Computing Lab I:

- Hands-on lab session: parallelising a realistic analytics workload on your own computer, measuring runtimes across worker counts, and plotting your first speedup curves.
- Detailed lab notebook provided; practice on your own PC.

- Lab Report 2 (individual) is assigned in this session; submission details will be announced in class.

SESSION 22 (LIVE IN-PERSON)

Topics:

- Performance of parallel programs: speedup, efficiency, and Amdahl's law.
- Strong and weak scaling.
- Scaling experiments on the University's HPC cluster.

Required Reading:

- *Parallel Programming for Multicore and Cluster Systems: Chapter 4, selected sections (See Bibliography)*

SESSION 23 (LIVE IN-PERSON)

Topics:

- Processing datasets larger than memory with Dask.
- Task graphs and lazy evaluation.
- Partitioned DataFrames, and how file formats and partitioning work together.

Required Reading:

- *Dask documentation: "10 Minutes to Dask" and the DataFrame tutorial (See Bibliography)*

SESSION 24 (LIVE IN-PERSON)

Topics:

- Distributed computing with Dask on the University's HPC cluster: schedulers, workers, and dashboards.
- MapReduce as a programming model and its modern descendants.
- Verifying that parallel programs are really parallel.

Required Reading:

- *Dask documentation: distributed scheduling overview (See Bibliography)*

SESSION 25 (LIVE IN-PERSON)

Group Project Review:

- Each team briefly presents its progress and current obstacles.
- Discussion on the project and doubts: implementation, debugging, and design decisions.
- Guidelines on how to present problem, parallelisation strategy, measured results, and the additional technique it explored beyond the course material.

SESSION 26 (LIVE IN-PERSON)

Topics:

- GPU computing for data analytics: why GPUs are fast, and when they are not.
- GPU programming from Python on the cluster's GPUs.
- A look under the hood: how GPU code differs from CPU code.

Required Reading:

- *High Performance Python: GPU sections (See Bibliography)*
- *CuPy documentation: "Basics of CuPy" (See Bibliography)*

SESSION 27 (LIVE IN-PERSON)

Topics:

- Beyond this course: the wider landscape of parallel and distributed computing for analytics, building on the techniques explored in the project presentations.
- Research opportunity at IE in this field.

Sessions review:

- Kahoot in class simulating exam questions.

- Doubts and discussions for the exam and project presentations.

SESSION 28 (LIVE IN-PERSON)

Final Exam

SESSIONS 29 - 30 (LIVE IN-PERSON)

Group Projects Presentations

- Group Project Presentation (Parallel Data Analytics on the HPC Cluster).
- Each team presents its problem, parallelization strategy, measured results, and the additional technique it explored beyond the course material.

EVALUATION CRITERIA

Disclaimer: The assessment method may be subject to modifications. Should changes occur, students will be notified on the first day of class and provided with a written document detailing the modifications.

criteria	percentage	Learning Objectives	Comments
Class Participation	10 %		
Individual Work	20 %		
Group Presentation	30 %		
Intermediate Tests	20 %		
Final Exam	20 %		

RE-SIT / RE-TAKE POLICY

Each student has four chances to pass any given course distributed over two consecutive academic years: ordinary call exams and extraordinary call exams (re-sits) in June/July. Students who do not comply with the 80% attendance rule during the semester will fail both calls for this Academic Year (ordinary and extraordinary) and have to re-take the course (i.e., re-enroll) in the next Academic Year.

Evaluation criteria:

- Students failing the course in the ordinary call (during the semester) will have to re-sit the exam in June / July (except those not complying with the attendance rule, who will not have that opportunity and must directly re-enroll in the course on the next Academic Year).

- The extraordinary call exams in June / July (re-sits) require your physical presence at the campus you are enrolled in (Segovia or Madrid). There is no possibility to change the date, location or format of any exam, under any circumstances. Dates and location of the June / July re-sit exams will be posted in advance. Please take this into consideration when planning your summer.

- The June / July re-sit exam will consist of a comprehensive exam. Your final grade for the course will depend on the performance in this exam only; continuous evaluation over the semester will not be taken into consideration. Students will have to achieve the minimum passing grade of 5 and can obtain a maximum grade of 8.0 (out of 10.0) – i.e., “notable” in the in the re-sit exam.

Retakers:

- Students who failed the subject on a previous Academic Year and are now re-enrolled as re-takers in a course will be needed to check the syllabus of the assigned professor, as well as contact the professor individually, regarding the specific evaluation criteria for them as retakers in the course during that semester (ordinary call of that Academic Year). The maximum grade that may be obtained in the retake exam (3rd call) is 10.0.

- After ordinary and extraordinary call exams are graded by the professor, you will have a possibility to attend a review session for that exam and course grade. Please be available to attend the session in order to clarify any concerns you might have regarding your exam. Your professor will inform you about the time and place of the review session. Any grade appeals require that the student attended the review session prior to appealing.

Students failing more than 18 ECTS (single degree) or 21 ECTS (dual degree) in the academic year after the June-July re-sits will be asked to leave the Program. Please, make sure to prepare yourself well for the exams in order to pass your failed subjects.

In case you decide to skip the opportunity to re-sit for an exam during the June / July extraordinary call, you will need to enroll in that course again for the next Academic Year as a re-taker and pay the corresponding extra cost. As you know, students have a total of four allowed calls to pass a given subject or course, in order to remain in the program.

BIBLIOGRAPHY

Compulsory

- Remzi H. Arpaci-Dusseau; Andrea C. Arpaci-Dusseau. (2018). *Operating Systems: Three Easy Pieces.* Arpaci-Dusseau Books, LLC Madison, WI, USA. ISBN 198508659X (Digital) Available free of charge at https://ostep.org null

- Micha Gorelick; Ian Ozsvald . (2025). *High Performance Python: Practical Performant Programming for Humans.* 3rd. O'Reilly Media. ISBN 1098165969 (Digital) null

Recommended

- William Shotts. (2019). *The Linux Command Line: A Complete Introduction.* 2. No Starch Press. ISBN 9781593279523 (Digital)

- Andrew Tanenbaum. (2023). *Modern Operating Systems.* 5th. Pearson Education Limited. ISBN 9781292459660 (Digital) null

- Dask Dev Team. (0). *Dask documentation.* ISBN 0308692446 (Digital) https://docs.dask.org/en/stable/index.html

- CuPy Dev Team. (0). *CuPy – NumPy & SciPy for GPU.* ISBN 0209809515 (Digital) https://docs.cupy.dev/en/stable/

- Thomas Rauber; Gudula Rünger. (2023). *Parallel Programming: for Multicore and Cluster Systems.* 3rd edition. Springer. ISBN 9783031289231 (Digital) null

BEHAVIOR RULES

Please, check the University's Academic Integrity Policy here. The Program Director may provide further indications.

ATTENDANCE POLICY

Please, check the University's Attendance Policy here. The Program Director may provide further indications.

ETHICAL POLICY

Please, check the University's Student Code of conduct here. The Program Director may provide further indications.